

DATA & AI PORTFOLIO · PROJECT NOTEBOOK 08

AutoClaim Intelligence Platform

Document AI, computer vision, and what happens when good stages compose

Author Nikhil Sinha

Repository github.com/Nikhil201716/08-AutoClaim-Intelligence-Platform

Stack Python · OCR · PyTorch · DuckDB · dbt · Ollama (qwen2.5:0.5b) · Streamlit

Edition 1.0

How to read this book

This book is written to be read from the beginning by someone who has not seen the project and does not yet know the techniques it uses. It is not API documentation - it is closer to a course text built around one real working system.

Two conventions run throughout. Callouts marked **WHY** explain a design decision and name the alternative that was rejected; callouts marked **WARNING** mark a trap that is easy to fall into and hard to notice afterwards. There is a chapter of pure terminal commands so the project can be reproduced without reading anything else, and a chapter of exercises with worked solutions.

Every code listing is extracted from the repository at the moment this book is built, so nothing here can drift out of step with the code it describes. If a listing looks wrong, the code is wrong.

All data in this project is synthetic. No real customer, transaction, or company data appears anywhere.

Contents

1. The Business Problem

- 1.1 A claim is a document, a photograph, and a disagreement
- 1.2 What was built and measured
- 1.3 The result the project is really about

2. Reading a Document

- 2.1 The stages between a PDF and a database row
- 2.2 Two ways to turn text into fields

3. A Comparison That Was Rigged, and Why That Matters

- 3.1 The measured result
- 3.2 Why the regex could not have lost
- 3.3 The experiment that would settle it

4. A Guided Tour of the Codebase

- 4.1 How to read this tour
- 4.2 `scripts/generate_claims_dataset.py`
- 4.3 `document_ai/ocr_extract.py`
- 4.4 `document_ai/regex_extraction.py`
- 4.5 `document_ai/llm_field_extraction.py`
- 4.6 `document_ai/evaluate_extraction.py`
- 4.7 `vision/train_damage_classifier.py`
- 4.8 `vision/predict.py`
- 4.9 `agents/claim_reconciliation_agent.py`
- 4.10 `scripts/evaluate_reconciliation.py`
- 4.11 `scripts/load_raw_to_duckdb.py`
- 4.12 `dashboard/streamlit_app.py`

5. The Damage Classifier

- 5.1 The task and the constraint
- 5.2 What a convolution actually does

5.3 83.9% accuracy, and what it conceals

5.4 The errors have a direction

6. Cross-Modal Reconciliation, and Where Its Errors Come From

6.1 The check

6.2 Perfect recall, and what it is worth

6.3 Tracing the false positives

6.4 The actual cause

7. The Semantic Layer and Data Contracts

7.1 AI outputs are data, and data needs governance

7.2 Lineage, and the question it answers

8. Worked Examples

8.1 Example 1 - A correctly flagged claim

8.2 Example 2 - A false positive, and its real cause

8.3 Example 3 - Compounding stage accuracy

8.4 Example 4 - Reading a perfect score

9. Running the Project, Step by Step

9.1 Prerequisites

9.2 Step 1 - Clone and enter the repository

9.3 Step 2 - Create an isolated environment

9.4 Step 3 - Install dependencies

9.5 Step 4 - Run the pipeline, stage by stage

9.6 Run everything in one command

9.7 Reproduce the false-positive attribution

9.8 Inspect the extraction that caused CLAIM_10002

9.9 Compare the two extractors directly

9.10 Launch the adjuster's console

9.11 Command reference card

9.12 Troubleshooting

10. The Adjuster's Console

10.1 One claim, four sources of evidence

10.2 Why the explanation is generated and the decision is not

11. Results, and What They Do Not Establish

11.1 The numbers

11.2 The four findings worth defending

11.3 What this does not establish

12. Exercises, with Solutions

12.1 Exercises

12.2 Solutions

13. Appendix

13.1 A - Repository map

13.2 B - Glossary

13.3 C - Reproducibility

13.4 D - Where this leads next

CHAPTER 1

The Business Problem

Why a claim is a multimodal document, what an adjuster actually does, and why the interesting work is the comparison rather than either half.

1.1 A claim is a document, a photograph, and a disagreement

An insurance claim arrives as a form and a photograph. The form says what happened and how much is being asked for. The photograph shows the damage. An adjuster's job is not to read either one - it is to check whether they agree.

That is a genuinely multimodal task, and it cannot be done by a document pipeline or a vision model alone. Each can tell you what its own input says. Only the comparison tells you whether the claim makes sense.

WHICH IS WHY THE RECONCILIATION AGENT IS THE POINT OF THE PROJECT

Extracting fields from a PDF is a solved problem. Classifying damage severity from a photograph is a well-understood one. Bolting them together produces two pipelines and no product.

The value appears at the join: *a severely damaged item with a trivial claim, or a pristine item with a large one, is worth a human looking at.* That is a cross-modal consistency check, and it is the thing an insurer would actually pay for.

1.2 What was built and measured

1.1 The five layers and their measured results

Layer	What it does	Headline
Document AI	OCR, then regex and LLM field extraction	regex 1.000, LLM 0.975
Vision	CNN damage classifier, trained from scratch	83.9% accuracy
Reconciliation	cross-modal consistency check	recall 1.000, precision 0.655
Semantic layer	dbt models, contracts, lineage	-
Console	one screen per claim for an adjuster	-

TWO OF THOSE THREE NUMBERS ARE EXACTLY 1.000, AND THAT IS THE FIRST THING TO INVESTIGATE

The portfolio's standing rule is that a suspiciously good result is a bug report. A regex scoring perfectly on all six fields, and a reconciliation agent missing zero inconsistent claims, are both results about the *test* as much as about the system.

Chapter 4 takes the regex result apart and Chapter 8 takes the recall apart. Neither is fraudulent; both are considerably less impressive than they look.

1.3 The result the project is really about

Reconciliation precision is 65.5% - about a third of flagged claims are legitimate. The evaluation report attributes 2 of the 10 false positives to vision misclassification.

1.2 Where the false positives actually come from

False positives	Count	Implication
Vision model was wrong	2	fixing the CNN helps
Vision model was correct	8	fixing the CNN changes nothing
Total	10	

THE OBVIOUS FIX ADDRESSES 20% OF THE PROBLEM

Faced with 65.5% precision, the natural response is to improve the weakest-looking component - the 83.9% vision model. That would address **two** of the ten false positives.

The other eight arrive with a correct severity, a legitimate claim, and a sound flagging rule. Chapter 8 traces them to their actual cause, which is upstream and invisible from the reconciliation stage's own metrics.

CHAPTER 2

Reading a Document

OCR, the pipeline from pixels to fields, and the two extraction strategies this project compares.

2.1 The stages between a PDF and a database row

3.1 Document intelligence, stage by stage

Stage	Input	Output	Failure mode
Rasterise	PDF	image	resolution too low to read
OCR	image	raw text	characters misread
Extract	raw text	named fields	right text, wrong field
Normalise	field strings	typed values	'\$1,234.00' parsed as 1.0
Validate	typed values	accepted or rejected	silently accepting nonsense

EACH STAGE INHERITS EVERY ERROR FROM THE ONE BEFORE IT

A character misread by OCR cannot be recovered by a better extractor - it can only be detected. A field extracted from the wrong line produces a perfectly well-formed number that is simply about something else, and no downstream validation on type or range will catch it.

This is the theme the whole book returns to. Chapter 8 measures what it costs here.

The OCR stage

document_ai/ocr_extract.py

```

"""
ocr_extract.py
-----
Extracts raw text from every claim PDF - the "OCR" step of this pipeline.

Honesty note on the word "OCR": these PDFs have embedded, selectable text
(they were generated with reportlab, not scanned from paper), so
`pdfplumber` reads that text directly rather than doing true image-based
optical character recognition. The DOWNSTREAM problem this project
demonstrates - turning inconsistent, semi-structured raw text into
validated structured fields (document_ai/llm_field_extraction.py) - is
the same problem either way; true image OCR (Tesseract or similar) would
slot in as a drop-in replacement for this module's `extract_text()`
function without changing anything downstream. Being upfront about this
distinction matters more than pretending otherwise.

Output: data/ocr_raw_text.json ({claim_id: raw_text})
"""

import json
from pathlib import Path

import pdfplumber

ROOT = Path(__file__).resolve().parent.parent
PDF_DIR = ROOT / "data" / "claim_pdfs"
OUT_PATH = ROOT / "data" / "ocr_raw_text.json"

def extract_text(pdf_path: Path) -> str:
    with pdfplumber.open(pdf_path) as pdf:
        return "\n".join(page.extract_text() or "" for page in pdf.pages)

def main():
    raw_texts = {}
    pdf_files = sorted(PDF_DIR.glob("*.pdf"))
    for pdf_path in pdf_files:
        claim_id = pdf_path.stem
        raw_texts[claim_id] = extract_text(pdf_path)

    with open(OUT_PATH, "w", encoding="utf-8") as f:
        json.dump(raw_texts, f, indent=2)

    print(f"Extracted text from {len(raw_texts)} PDFs -> {OUT_PATH}")
    sample_id = next(iter(raw_texts))
    print(f"\nSample ({sample_id}): \n{raw_texts[sample_id]}")

if __name__ == "__main__":
    main()

```

2.2 Two ways to turn text into fields

Once you have raw text, there are two strategies, and this project implements both so they can be compared on the same 80 documents.

3.2 Regex versus LLM extraction

	Regex	LLM
How it works	pattern matching on known layout	the model reads and reports
Setup cost	write a pattern per field	write a prompt
Speed	microseconds	seconds
Deterministic	yes	no
Handles layout variation	no	yes
Handles a new field	new pattern needed	edit the prompt
Fails	loudly - no match	quietly - a plausible wrong value

Regex extraction

document_ai/regex_extraction.py

```

"""
regex_extraction.py
-----

The baseline every "AI-powered" document extraction claim has to beat:
plain regex over the OCR'd text. Since claim forms are still fundamentally
"Label: Value" pairs (just with label wording/order variance across the 3
layout templates - see scripts/generate_claims_dataset.py), a
hand-maintained set of label aliases can plausibly do quite well here -
and it's worth honestly finding out whether it does, rather than assuming
an LLM is automatically the better tool (the same "know which tool
actually fits" judgment applied throughout this portfolio).

Output: data/regex_extracted_fields.json
"""

import json
import re
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RAW_TEXT_PATH = ROOT / "data" / "ocr_raw_text.json"
OUT_PATH = ROOT / "data" / "regex_extracted_fields.json"

# Each field maps to the label variants actually used across the 3 layout
# templates in scripts/generate_claims_dataset.py.
FIELD_PATTERNS = {
    "claim_id": [r"Claim ID:\s*(\S+)"],
    "policyholder": [r"Policyholder Name:\s*(.+)"],
    "policy_number": [r"Policy Number:\s*(\S+)"],
    "claim_date": [r>Date of Claim:\s*(\S+)"],
    "category": [r"Item Category:\s*(.+)"],
    "item_description": [r"Item Description:\s*(.+)"],
    "item_value": [r"(?:Item Value|Estimated Item Value):\s*\$?([\d,]+\.\d{2})"],
    "claimed_amount": [r"(?:Claimed Amount|Amount Claimed \($\)):\s*\$?([\d,]+\.\d{2})"],
}

def extract_fields(text: str) -> dict:
    result = {}
    for field, patterns in FIELD_PATTERNS.items():
        value = None
        for pattern in patterns:
            m = re.search(pattern, text)
            if m:
                value = m.group(1).strip()
                break
        result[field] = value
    return result

def main():
    with open(RAW_TEXT_PATH, encoding="utf-8") as f:
        raw_texts = json.load(f)

    extracted = {claim_id: extract_fields(text) for claim_id, text in raw_texts.items()}

```

LLM extraction

document_ai/llm_field_extraction.py

```

"""
llm_field_extraction.py
-----
Structured field extraction via a local Ollama model, asked to return
JSON directly - the "AI-powered" half of the document intelligence
comparison against document_ai/regex_extraction.py's hand-maintained
label-alias baseline.

GUARDRAIL: the model's response is only accepted if it parses as valid
JSON containing all 8 expected keys. If it doesn't (malformed JSON,
missing keys, extra prose wrapped around the JSON), this module does NOT
guess or partially accept it - the claim is marked `extraction_failed`
for human review, the same never-silently-accept-a-bad-parse principle
used for every guardrail elsewhere in this portfolio.

Output: data/llm_extracted_fields.json
"""

import argparse
import json
import re
from pathlib import Path

import ollama

ROOT = Path(__file__).resolve().parent.parent
RAW_TEXT_PATH = ROOT / "data" / "ocr_raw_text.json"
OUT_PATH = ROOT / "data" / "llm_extracted_fields.json"
OLLAMA_MODEL = "qwen2.5:0.5b"

EXPECTED_KEYS = ["claim_id", "policyholder", "policy_number", "claim_date",
                 "category", "item_description", "item_value", "claimed_amount"]

PROMPT_TEMPLATE = """Extract fields from this insurance claim form text into JSON.

Claim form text:
---
{text}
---

Respond with ONLY a JSON object, no other text. The JSON MUST use exactly these 8 key names, \
spelled exactly like this, even if the form uses different wording for the label - map the form's \
label to the correct key below, do not invent your own key names:

{{
  "claim_id": "...",
  "policyholder": "...",
  "policy_number": "...",
  "claim_date": "...",
  "category": "...",
  "item_description": "...",
  "item_value": <number, no $ or commas>,
  "claimed_amount": <number, no $ or commas>
}}

```

THE LAST ROW IS THE ONE THAT DECIDES ARCHITECTURE

A regex that does not match returns nothing, and nothing is easy to detect and route to a human. An LLM that misreads a field returns a confident, well-formatted, plausible value that flows straight into the warehouse.

Loud failure is worth a great deal of accuracy. This is the same argument Project 4 makes about a missing source file raising rather than returning empty - and Chapter 8 shows exactly what a quiet extraction failure costs three stages downstream.

CHAPTER 3

A Comparison That Was Rigged, and Why That Matters

The regex scored 1.000 and the LLM 0.975. The regex did not win - it was scoring its own template.

3.1 The measured result

4.1 Extraction accuracy by field, from the real evaluation

Field	Regex	LLM
policyholder	1.000	0.9875
policy_number	1.000	1.000
category	1.000	1.000
item_description	1.000	1.000
item_value	1.000	0.900
claimed_amount	1.000	0.9625
Overall	1.0000	0.9750

Read naively: the regex is better, it is faster, it is deterministic, and the LLM has bought nothing. That reading is wrong, and the reason is worth more than the numbers.

3.2 Why the regex could not have lost

The claim PDFs are generated by `scripts/generate_claims_dataset.py` from a fixed template. The regex patterns were written against that template. Every document the regex is tested on has exactly the layout the regex was built for.

THE REGEX IS BEING SCORED AGAINST ITS OWN GENERATOR

This is circular. The regex encodes the template; the documents are produced by the template; therefore the regex matches. A score of 1.0000 was determined the moment the pattern was written, and no amount of test data of this kind could have produced any other number.

It is not cheating - it is the natural consequence of testing template-matching against a template. But it means the comparison measures **how well each method handles a document it has already been fitted to**, which is not the question anyone actually has.

What the LLM's 0.975 is measuring instead

The LLM was not shown the template. It reads text and reports fields, and it gets 97.5% of them right without having been told where to look. That number is measuring something real, and it is being compared against a number that is measuring nothing.

3.3 The experiment that would settle it

The comparison becomes meaningful the moment the documents stop matching the template. Three variations, in increasing order of realism:

1. **Reorder the fields.** Swap the position of item value and claimed amount in the layout. A positional regex breaks; the LLM is largely unaffected.
2. **Vary the labels.** 'Item Value', 'Declared Value', 'Value of Item'. Every variant needs a new alternative in the pattern; the LLM reads them as the same concept.
3. **Introduce real OCR noise.** Scan artefacts, skew, a character misread. The regex fails to match at all; the LLM often recovers from context.

THE PREDICTION, STATED IN ADVANCE

Under any of those three, the regex's accuracy falls sharply and the LLM's falls slowly. **The crossover point - how much layout variation it takes for the LLM to win - is the actual finding this comparison should have produced**, and it is not in this project.

Stating the prediction before running it is what makes it a test rather than a story. Project 3 did the same with its windowing hypothesis - and that one was *refuted*, which is precisely why the discipline is worth keeping.

WHAT THE HONEST CONCLUSION IS TODAY

On documents matching a known fixed template, use the regex - it is faster, deterministic, free, and fails loudly. That conclusion is genuinely supported.

The measured 1.000 says nothing about documents that do not match the template, which is the case every production system actually faces. A real deployment would use the regex as the fast path and the LLM as the fallback when it fails to match - which uses each method where its measured strength applies.

CHAPTER 4

A Guided Tour of the Codebase

Every module that matters, in the order the pipeline runs them, with its purpose and public surface. This chapter is generated from the repository, so it cannot describe a function that no longer exists.

4.1 How to read this tour

Read in pipeline order - generation, document AI, vision, then reconciliation. The evaluation scripts come last because each one grades a stage that has already run, and the reconciliation evaluator is the only place the whole chain's behaviour becomes visible.

4.2 scripts/generate_claims_dataset.py

Why it exists. The seeded generator: claim records, rendered PDFs, damage images, and the deliberately inconsistent claims the reconciliation agent is graded against.

What the module says about itself. Generates the shared synthetic dataset both halves of this platform run against: for each claim, a PDF claim form (for document_ai/ to extract from) AND a paired damage photo (for vision/ to classify) - both derived from the same ground-truth record, which is what makes cross-modal reconciliation (agents/claim_reconciliation_agent.py) meaningful later: it's checking whether two independently-built AI pipelines agree about the same real underlying claim.

Public functions in scripts/generate_claims_dataset.py

Function	What it does
make_claim_record(i)	—
render_pdf(record, path)	A deliberately imperfect claim form - field order and phrasing vary
render_damage_image(severity, path, size)	Parametric synthetic damage photo: a product silhouette with

Opening of the module

scripts/generate_claims_dataset.py

```

"""
generate_claims_dataset.py
-----

Generates the shared synthetic dataset both halves of this platform run
against: for each claim, a PDF claim form (for document_ai/ to extract
from) AND a paired damage photo (for vision/ to classify) - both derived
from the same ground-truth record, which is what makes cross-modal
reconciliation (agents/claim_reconciliation_agent.py) meaningful later:
it's checking whether two independently-built AI pipelines agree about
the same real underlying claim.

DELIBERATE INJECTED SCENARIO: for ~18% of claims, the claimed dollar
amount is set INCONSISTENT with the claim's true damage severity (e.g. a
high claim amount on a cosmetically-minor item, or a low claim amount on
a severely-damaged one) - a plausible mix of data-entry error and
potential-fraud signal. Neither the document extraction pipeline nor the
vision pipeline is told which claims these are; only
scripts/evaluate_reconciliation.py checks detection against this ground
truth, never the reconciliation agent's own logic.

Also deliberately messy: claim forms vary in layout/field order/phrasing
(not one clean template) and item categories/descriptions have realistic
wording variance - because a document-extraction pipeline that only works
on one exact template isn't demonstrating anything.

No real claimant or insurance data is used anywhere in this project.

Output:
  data/claims_ground_truth.csv
  data/claim_pdfs/CLAIM_*.pdf
  data/damage_images/{none,minor,moderate,severe}/CLAIM_*.png
"""

import random
from pathlib import Path
from datetime import datetime, timedelta

import numpy as np
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from PIL import Image, ImageDraw

SEED = 11
rng = np.random.default_rng(SEED)
random.seed(SEED)

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
PDF_DIR = DATA_DIR / "claim_pdfs"
IMG_DIR = DATA_DIR / "damage_images"
for d in [PDF_DIR] + [IMG_DIR / c for c in ["none", "minor", "moderate", "severe"]]:
    d.mkdir(parents=True, exist_ok=True)

N_CLAIMS = 320
MISMATCH_RATE = 0.18

CATEGORIES = {

```

```

    "Electronics": [("Laptop", 900), ("Smart TV", 650), ("Tablet", 350), ("Camera", 500)],
    "Appliance": [("Refrigerator", 1200), ("Washing Machine", 700), ("Microwave", 180)],
    "Furniture": [("Sofa", 800), ("Dining Table", 450), ("Office Chair", 220)],
    "Vehicle Part": [("Windshield", 400), ("Bumper", 550), ("Side Mirror", 150)],
}
FIRST_NAMES = ["James", "Maria", "Robert", "Linda", "David", "Sarah", "Michael", "Patricia",
               "John", "Jennifer", "Carlos", "Aisha", "Wei", "Priya", "Liam", "Emma"]
LAST_NAMES = ["Smith", "Garcia", "Johnson", "Williams", "Brown", "Davis", "Martinez", "Lee",
              "Wilson", "Anderson", "Thomas", "Patel", "Kim", "Nguyen", "Clark", "Rodriguez"]

SEVERITY_LEVELS = ["none", "minor", "moderate", "severe"]
SEVERITY_WEIGHTS = [0.10, 0.35, 0.35, 0.20]
# Roughly what a "reasonable" claim amount looks like per severity - a
# fraction of item value, with some natural variance.
SEVERITY_CLAIM_FRACTION = {"none": (0.0, 0.05), "minor": (0.08, 0.25),

```

4.3 document_ai/ocr_extract.py

Why it exists. PDF to raw text. The first stage, and the one whose errors nothing downstream can recover.

What the module says about itself. Extracts raw text from every claim PDF - the "OCR" step of this pipeline.

Public functions in document_ai/ocr_extract.py

Function	What it does
extract_text(pdf_path)	—
main()	—

Opening of the module

document_ai/ocr_extract.py

```

"""
ocr_extract.py
-----
Extracts raw text from every claim PDF - the "OCR" step of this pipeline.

Honesty note on the word "OCR": these PDFs have embedded, selectable text
(they were generated with reportlab, not scanned from paper), so
`pdfplumber` reads that text directly rather than doing true image-based
optical character recognition. The DOWNSTREAM problem this project
demonstrates - turning inconsistent, semi-structured raw text into
validated structured fields (document_ai/llm_field_extraction.py) - is
the same problem either way; true image OCR (Tesseract or similar) would
slot in as a drop-in replacement for this module's `extract_text()`
function without changing anything downstream. Being upfront about this
distinction matters more than pretending otherwise.

Output: data/ocr_raw_text.json ({claim_id: raw_text})
"""

import json
from pathlib import Path

import pdfplumber

ROOT = Path(__file__).resolve().parent.parent
PDF_DIR = ROOT / "data" / "claim_pdfs"
OUT_PATH = ROOT / "data" / "ocr_raw_text.json"

def extract_text(pdf_path: Path) -> str:
    with pdfplumber.open(pdf_path) as pdf:
        return "\n".join(page.extract_text() or "" for page in pdf.pages)

def main():
    raw_texts = {}
    pdf_files = sorted(PDF_DIR.glob("*.pdf"))
    for pdf_path in pdf_files:
        claim_id = pdf_path.stem
        raw_texts[claim_id] = extract_text(pdf_path)

    with open(OUT_PATH, "w", encoding="utf-8") as f:
        json.dump(raw_texts, f, indent=2)

    print(f"Extracted text from {len(raw_texts)} PDFs -> {OUT_PATH}")
    sample_id = next(iter(raw_texts))
    print(f"\nSample ({sample_id}): \n{raw_texts[sample_id]}")

if __name__ == "__main__":
    main()

```

4.4 document_ai/regex_extraction.py

Why it exists. Pattern-based field extraction against the known template - fast, deterministic, and fitted to the layout it is tested on.

What the module says about itself. The baseline every "AI-powered" document extraction claim has to beat: plain regex over the OCR'd text. Since claim forms are still fundamentally "Label: Value" pairs (just with label wording/order variance across the 3 layout templates - see `scripts/generate_claims_dataset.py`), a hand-maintained set of label aliases can plausibly do quite well here - and it's worth honestly finding out whether it does, rather than assuming an LLM is automatically the better tool (the same "know which tool actually fits" judgment applied throughout this portfolio).

Public functions in `document_ai/regex_extraction.py`

Function	What it does
<code>extract_fields(text)</code>	—
<code>main()</code>	—

Opening of the module

document_ai/regex_extraction.py

```

"""
regex_extraction.py
-----

The baseline every "AI-powered" document extraction claim has to beat:
plain regex over the OCR'd text. Since claim forms are still fundamentally
"Label: Value" pairs (just with label wording/order variance across the 3
layout templates - see scripts/generate_claims_dataset.py), a
hand-maintained set of label aliases can plausibly do quite well here -
and it's worth honestly finding out whether it does, rather than assuming
an LLM is automatically the better tool (the same "know which tool
actually fits" judgment applied throughout this portfolio).

Output: data/regex_extracted_fields.json
"""

import json
import re
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RAW_TEXT_PATH = ROOT / "data" / "ocr_raw_text.json"
OUT_PATH = ROOT / "data" / "regex_extracted_fields.json"

# Each field maps to the label variants actually used across the 3 layout
# templates in scripts/generate_claims_dataset.py.
FIELD_PATTERNS = {
    "claim_id": [r"Claim ID:\s*(\S+)"],
    "policyholder": [r"Policyholder Name:\s*(.+)"],
    "policy_number": [r"Policy Number:\s*(\S+)"],
    "claim_date": [r>Date of Claim:\s*(\S+)"],
    "category": [r"Item Category:\s*(.+)"],
    "item_description": [r"Item Description:\s*(.+)"],
    "item_value": [r"(?:Item Value|Estimated Item Value):\s*\$?([\d,]+\.\d{2})"],
    "claimed_amount": [r"(?:Claimed Amount|Amount Claimed \($\)):\s*\$?([\d,]+\.\d{2})"],
}

def extract_fields(text: str) -> dict:
    result = {}
    for field, patterns in FIELD_PATTERNS.items():
        value = None
        for pattern in patterns:
            m = re.search(pattern, text)
            if m:
                value = m.group(1).strip()
                break
        result[field] = value
    return result

def main():
    with open(RAW_TEXT_PATH, encoding="utf-8") as f:
        raw_texts = json.load(f)

    extracted = {claim_id: extract_fields(text) for claim_id, text in raw_texts.items()}

    with open(OUT_PATH, "w", encoding="utf-8") as f:

```

```

    json.dump(extracted, f, indent=2)

    n_complete = sum(1 for v in extracted.values() if all(v.values()))
    print(f"Regex-extracted fields for {len(extracted)} claims -> {OUT_PATH}")
    print(f"Claims with all fields found: {n_complete}/{len(extracted)}")

if __name__ == "__main__":
    main()

```

4.5 document_ai/llm_field_extraction.py

Why it exists. The local model reads the text and reports fields, without being told the layout.

What the module says about itself. Structured field extraction via a local Ollama model, asked to return JSON directly - the "AI-powered" half of the document intelligence comparison against document_ai/regex_extraction.py's hand-maintained label-alias baseline.

Public functions in document_ai/llm_field_extraction.py

Function	What it does
extract_fields_llm(text)	—
main()	—

Opening of the module

document_ai/llm_field_extraction.py

```

"""
llm_field_extraction.py
-----
Structured field extraction via a local Ollama model, asked to return
JSON directly - the "AI-powered" half of the document intelligence
comparison against document_ai/regex_extraction.py's hand-maintained
label-alias baseline.

GUARDRAIL: the model's response is only accepted if it parses as valid
JSON containing all 8 expected keys. If it doesn't (malformed JSON,
missing keys, extra prose wrapped around the JSON), this module does NOT
guess or partially accept it - the claim is marked `extraction_failed`
for human review, the same never-silently-accept-a-bad-parse principle
used for every guardrail elsewhere in this portfolio.

Output: data/llm_extracted_fields.json
"""

import argparse
import json
import re
from pathlib import Path

import ollama

ROOT = Path(__file__).resolve().parent.parent
RAW_TEXT_PATH = ROOT / "data" / "ocr_raw_text.json"
OUT_PATH = ROOT / "data" / "llm_extracted_fields.json"
OLLAMA_MODEL = "qwen2.5:0.5b"

EXPECTED_KEYS = ["claim_id", "policyholder", "policy_number", "claim_date",
                 "category", "item_description", "item_value", "claimed_amount"]

PROMPT_TEMPLATE = """Extract fields from this insurance claim form text into JSON.

Claim form text:
---
{text}
---

Respond with ONLY a JSON object, no other text. The JSON MUST use exactly these 8 key names, \
spelled exactly like this, even if the form uses different wording for the label - map the form's \
label to the correct key below, do not invent your own key names:

{{
  "claim_id": "...",
  "policyholder": "...",
  "policy_number": "...",
  "claim_date": "...",
  "category": "...",
  "item_description": "...",
  "item_value": <number, no $ or commas>,
  "claimed_amount": <number, no $ or commas>
}}

("Item Value" or "Estimated Item Value" in the form both map to the "item_value" key. "Claimed \
Amount" or "Amount Claimed ($)" in the form both map to the "claimed_amount" key.)

```

```
JSON:""
```

```
def _try_parse_json(raw: str) -> dict | None:
    # The model sometimes wraps JSON in ```json fences or adds a stray
    # sentence - pull out the first {...} block rather than requiring a
    # perfectly clean response.
    match = re.search(r"\{.*\}", raw, re.DOTALL)
    if not match:
        return None
    try:
        return json.loads(match.group(0))
    except json.JSONDecodeError:
        return None
```

4.6 document_ai/evaluate_extraction.py

Why it exists. Scores both extractors against ground truth, field by field.

What the module says about itself. Compares both document-extraction approaches - the regex baseline (document_ai/regex_extraction.py) and the LLM extraction (document_ai/llm_field_extraction.py) - against the real ground truth (data/claims_ground_truth.csv, generated alongside the PDFs and never seen by either extractor). Text fields are scored by exact match; numeric fields (item_value, claimed_amount) by a small tolerance ($\pm \$0.01$) to allow for legitimate float rounding, not sloppy scoring.

Public functions in document_ai/evaluate_extraction.py

Function	What it does
to_float(v)	—
score(extracted, truth)	—
main()	—

Opening of the module

document_ai/evaluate_extraction.py

```

"""
evaluate_extraction.py
-----
Compares both document-extraction approaches - the regex baseline
(document_ai/regex_extraction.py) and the LLM extraction
(document_ai/llm_field_extraction.py) - against the real ground truth
(data/claims_ground_truth.csv, generated alongside the PDFs and never
seen by either extractor). Text fields are scored by exact match;
numeric fields (item_value, claimed_amount) by a small tolerance ( $\pm 0.01$ )
to allow for legitimate float rounding, not sloppy scoring.

Only claims the LLM extraction actually ran on (see --sample-size in
llm_field_extraction.py) are included in the LLM comparison, so it's an
apples-to-apples comparison over the same claims, not the regex
baseline's full-dataset number against a partial LLM sample.

Output: reports/extraction_evaluation.json/.md
"""

import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)

TEXT_FIELDS = ["policyholder", "policy_number", "category", "item_description"]
NUMERIC_FIELDS = ["item_value", "claimed_amount"]
ALL_FIELDS = TEXT_FIELDS + NUMERIC_FIELDS

def to_float(v):
    try:
        return float(str(v).replace("$", "").replace(",", ""))
    except (TypeError, ValueError):
        return None

def score(extracted: dict, truth: dict) -> dict:
    field_correct = {}
    for field in TEXT_FIELDS:
        field_correct[field] = str(extracted.get(field, "")).strip().lower() == \
            str(truth.get(field, "")).strip().lower()
    for field in NUMERIC_FIELDS:
        ev, tv = to_float(extracted.get(field)), truth.get(field)
        field_correct[field] = ev is not None and tv is not None and abs(ev - tv) < 0.01
    return field_correct

def main():
    truth_df = pd.read_csv(DATA_DIR / "claims_ground_truth.csv").set_index("claim_id")

    with open(DATA_DIR / "regex_extracted_fields.json", encoding="utf-8") as f:
        regex_extracted = json.load(f)

```

```

with open(DATA_DIR / "llm_extracted_fields.json", encoding="utf-8") as f:
    llm_extracted = json.load(f)

results = {"regex": {f: 0 for f in ALL_FIELDS}, "llm": {f: 0 for f in ALL_FIELDS}}
n_regex, n_llm = 0, 0

for claim_id, extracted in regex_extracted.items():
    truth = truth_df.loc[claim_id].to_dict()
    correct = score(extracted, truth)
    for field, is_correct in correct.items():
        results["regex"][field] += int(is_correct)
    n_regex += 1

for claim_id, extracted in llm_extracted.items():
    if extracted.get("extraction_failed"):

```

4.7 vision/train_damage_classifier.py

Why it exists. The CNN, trained from scratch on 258 images, and the confusion matrix that reveals its directional bias.

What the module says about itself. Trains a small convolutional neural network, from scratch, on the synthetic damage photos (data/damage_images/{none,minor,moderate,severe}/) to classify damage severity - the computer vision half of this platform, kept deliberately small (3 conv blocks, 96x96 grayscale-ish RGB inputs) so it trains in minutes on CPU with no GPU and no pretrained-model download, consistent with every other model choice in this portfolio (no heavy framework where a lighter one does the job on this hardware).

Classes in vision/train_damage_classifier.py

Class	Purpose	Key methods
DamageCNN	—	forward

Public functions in vision/train_damage_classifier.py

Function	What it does
load_dataset()	—
stratified_split(paths, labels)	—
load_image_tensor(path)	—
batches(paths, labels, idx, batch_size)	—
main()	—

Opening of the module

vision/train_damage_classifier.py

```

"""
train_damage_classifier.py
-----
Trains a small convolutional neural network, from scratch, on the
synthetic damage photos (data/damage_images/{none,minor,moderate,severe}/)
to classify damage severity - the computer vision half of this platform,
kept deliberately small (3 conv blocks, 96x96 grayscale-ish RGB inputs)
so it trains in minutes on CPU with no GPU and no pretrained-model
download, consistent with every other model choice in this portfolio
(no heavy framework where a lighter one does the job on this hardware).

Uses `torchvision`-free image loading (plain PIL + a hand-rolled Dataset)
since torchvision isn't installed on this machine and isn't needed for
something this small.

Split: stratified 80/20 train/test by class, so evaluation reflects a
genuine holdout, not training-set performance.

Output:
  vision/artifacts/damage_classifier.pt - trained model weights
  reports/vision_evaluation.json/.md    - accuracy, confusion matrix, per-class P/R
"""

import json
from collections import defaultdict
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from PIL import Image

ROOT = Path(__file__).resolve().parent.parent
IMG_DIR = ROOT / "data" / "damage_images"
ARTIFACTS_DIR = ROOT / "vision" / "artifacts"
ARTIFACTS_DIR.mkdir(exist_ok=True)
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)

SEED = 5
torch.manual_seed(SEED)
rng = np.random.default_rng(SEED)

CLASSES = ["none", "minor", "moderate", "severe"]
CLASS_TO_IDX = {c: i for i, c in enumerate(CLASSES)}
IMG_SIZE = 96
TEST_FRACTION = 0.2
BATCH_SIZE = 16
N_EPOCHS = 25
LR = 1e-3

def load_dataset():
    paths, labels = [], []
    for cls in CLASSES:
        for p in sorted((IMG_DIR / cls).glob("*.png")):

```

```

        paths.append(p)
        labels.append(CLASS_TO_IDX[cls])
    return paths, np.array(labels)

def stratified_split(paths, labels):
    train_idx, test_idx = [], []
    for cls_idx in range(len(CLASSES)):
        idx = np.where(labels == cls_idx)[0]
        rng.shuffle(idx)
        n_test = max(1, int(len(idx) * TEST_FRACTION))
        test_idx.extend(idx[:n_test])
        train_idx.extend(idx[n_test:])
    rng.shuffle(train_idx)
    rng.shuffle(test_idx)

```

4.8 vision/predict.py

Why it exists. Inference over the claim images, producing the severities the reconciliation agent consumes.

What the module says about itself. Runs the trained damage-severity CNN (vision/artifacts/damage_classifier.pt) over every claim's damage photo and saves a prediction per claim - this is what agents/claim_reconciliation_agent.py and the dbt marts layer both consume, so the CV pipeline's real output feeds the rest of the platform rather than each part staying siloed.

Public functions in vision/predict.py

Function	What it does
load_image_tensor(path)	—
main()	—

Opening of the module

vision/predict.py

```

"""
predict.py
-----

Runs the trained damage-severity CNN (vision/artifacts/damage_classifier.pt)
over every claim's damage photo and saves a prediction per claim - this is
what agents/claim_reconciliation_agent.py and the dbt marts layer both
consume, so the CV pipeline's real output feeds the rest of the platform
rather than each part staying siloed.

Output: reports/vision_predictions.json ({claim_id: {predicted_severity, confidence}})
"""

import json
from pathlib import Path

import numpy as np
import torch
import torch.nn.functional as F
from PIL import Image

import sys
ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from vision.train_damage_classifier import DamageCNN, CLASSES, IMG_SIZE # noqa: E402

IMG_DIR = ROOT / "data" / "damage_images"
ARTIFACTS_DIR = ROOT / "vision" / "artifacts"
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)

def load_image_tensor(path) -> torch.Tensor:
    img = Image.open(path).convert("RGB").resize((IMG_SIZE, IMG_SIZE))
    arr = np.array(img, dtype=np.float32) / 255.0
    return torch.from_numpy(arr).permute(2, 0, 1)

def main():
    model = DamageCNN(n_classes=len(CLASSES))
    model.load_state_dict(torch.load(ARTIFACTS_DIR / "damage_classifier.pt"))
    model.eval()

    predictions = {}
    image_paths = []
    for cls_dir in IMG_DIR.iterdir():
        if cls_dir.is_dir():
            image_paths.extend(cls_dir.glob("*.png"))

    with torch.no_grad():
        for path in sorted(image_paths):
            claim_id = path.stem
            tensor = load_image_tensor(path).unsqueeze(0)
            logits = model(tensor)
            probs = F.softmax(logits, dim=1)[0]
            pred_idx = int(probs.argmax())
            predictions[claim_id] = {
                "predicted_severity": CLASSES[pred_idx],

```

```

        "confidence": round(float(probs[pred_idx]), 4),
    }

    with open(REPORTS_DIR / "vision_predictions.json", "w", encoding="utf-8") as f:
        json.dump(predictions, f, indent=2)

    print(f"Predicted severity for {len(predictions)} claims -> "
          f"{REPORTS_DIR / 'vision_predictions.json'}")

if __name__ == "__main__":
    main()

```

4.9 agents/claim_reconciliation_agent.py

Why it exists. The cross-modal check. Deterministic bands, an LLM used only to narrate the verdict it was handed.

What the module says about itself. The cross-modal check that makes this platform more than two AI pipelines bolted together: for every claim, does the CLAIMED DOLLAR AMOUNT (document_ai/ extraction) roughly match the DAMAGE SEVERITY the photo actually shows (vision/ classification)? A severe-looking photo with a trivial claim, or a pristine-looking photo with a huge claim, is exactly the kind of signal a claims adjuster would want surfaced automatically - whether the cause is a data-entry error or something worth a closer look.

Public functions in agents/claim_reconciliation_agent.py

Function	What it does
check_claim(severity, item_value, claimed_amount)	—
explain_flag(claim_id, severity, item_value, claimed_amount)	—
main()	—

Opening of the module

agents/claim_reconciliation_agent.py

```

"""
claim_reconciliation_agent.py
-----

The cross-modal check that makes this platform more than two AI pipelines
bolted together: for every claim, does the CLAIMED DOLLAR AMOUNT
(document_ai/ extraction) roughly match the DAMAGE SEVERITY the photo
actually shows (vision/ classification)? A severe-looking photo with a
trivial claim, or a pristine-looking photo with a huge claim, is exactly
the kind of signal a claims adjuster would want surfaced automatically -
whether the cause is a data-entry error or something worth a closer look.

DESIGN NOTE - deterministic decision, LLM narration, same as
agents/action_agent.py in Project 7: "is claimed_amount / item_value
inside a reasonable range for this severity" is a threshold check with a
clear right answer, not something to hand to a language model's
judgment. These expected-range bands are this agent's OWN independent
business-reasonableness judgment - NOT reverse-engineered from
scripts/generate_claims_dataset.py's generation logic, which the agent
never sees. The bands are deliberately a bit more permissive than the
generator's own (see comments below) so the agent has to actually clear
a real bar, not just replay the answer key.

A Local Ollama model is used only to turn an already-made flag decision
into a one-sentence explanation an adjuster can read in 5 seconds -
grounded in the specific numbers, not asked to make the call itself.

Output: reports/reconciliation_flags.json/.md
"""

import json
from pathlib import Path

import ollama
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)
OLLAMA_MODEL = "qwen2.5:0.5b"

# This agent's OWN reasonable-range judgment (a claims adjuster's rule of
# thumb), deliberately not copied from the data generator's bands.
EXPECTED_FRACTION_RANGE = {
    "none": (0.0, 0.10),
    "minor": (0.05, 0.30),
    "moderate": (0.25, 0.65),
    "severe": (0.60, 1.05),
}

}

EXPLAIN_PROMPT = """A claims-reconciliation check flagged a mismatch. The photo of the item was \
classified as "{severity}" damage. The claim form states the item is worth ${item_value:.2f} and \
${claimed_amount:.2f} is being claimed - that's {fraction:.0%} of the item's value, outside the \
{lo:.0%}-{hi:.0%} range normally expected for "{severity}" damage. In one sentence, explain this \
mismatch to a claims adjuster and suggest one next step. Reason only from the numbers given."""

```

```

def check_claim(severity: str, item_value: float, claimed_amount: float) -> dict:
    if item_value <= 0:
        return {"flagged": False, "reason": "item_value not available for reconciliation"}
    fraction = claimed_amount / item_value
    lo, hi = EXPECTED_FRACTION_RANGE.get(severity, (0.0, 1.0))
    flagged = not (lo <= fraction <= hi)
    return {"flagged": flagged, "fraction": round(fraction, 3), "expected_range": (lo, hi)}

def explain_flag(claim_id, severity, item_value, claimed_amount, fraction, lo, hi) -> str:
    prompt = EXPLAIN_PROMPT.format(severity=severity, item_value=item_value,
                                    claimed_amount=claimed_amount, fraction=fraction, lo=lo, hi=hi)

    response = ollama.chat(
        model=OLLAMA_MODEL,
        messages=[{"role": "user", "content": prompt}],

```

4.10 scripts/evaluate_reconciliation.py

Why it exists. Grades the agent against the injected inconsistencies and records, for each false positive, whether the vision model was correct - the data Chapter 8 rests on.

What the module says about itself. Checks agents/claim_reconciliation_agent.py's flags against the real ground truth (data/claims_ground_truth.csv's is_injected_mismatch column) - the ONLY script in this platform allowed to look at that column, and only for evaluation, never for the agent's own decision logic.

Opening of the module

scripts/evaluate_reconciliation.py

```

"""
evaluate_reconciliation.py
-----
Checks agents/claim_reconciliation_agent.py's flags against the real
ground truth (data/claims_ground_truth.csv's is_injected_mismatch column)
- the ONLY script in this platform allowed to look at that column, and
only for evaluation, never for the agent's own decision logic.

This measures something specific and important: the reconciliation
agent's flag rate (29 out of ~80 evaluated) is meaningfully higher than
the injected mismatch rate (18.8% of the full 320 claims) would predict.
This script exists to find out honestly why - is the agent catching real
injected mismatches (true positives) plus reasonable additional cases
from imperfect upstream extraction/CV, or is it mostly noise?

Output: reports/reconciliation_evaluation.json/.md
"""

import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
REPORTS_DIR = ROOT / "reports"

truth_df = pd.read_csv(DATA_DIR / "claims_ground_truth.csv").set_index("claim_id")

with open(DATA_DIR / "llm_extracted_fields.json", encoding="utf-8") as f:
    llm_extracted = json.load(f)
with open(REPORTS_DIR / "vision_predictions.json", encoding="utf-8") as f:
    vision_preds = json.load(f)
with open(REPORTS_DIR / "reconciliation_flags.json", encoding="utf-8") as f:
    flags = json.load(f)

flagged_ids = {f["claim_id"] for f in flags}

evaluated_ids = [
    cid for cid, fields in llm_extracted.items()
    if not fields.get("extraction_failed") and cid in vision_preds
]

tp = fp = fn = tn = 0
for claim_id in evaluated_ids:
    is_true_mismatch = bool(truth_df.loc[claim_id, "is_injected_mismatch"])
    is_flagged = claim_id in flagged_ids
    if is_true_mismatch and is_flagged:
        tp += 1
    elif not is_true_mismatch and is_flagged:
        fp += 1
    elif is_true_mismatch and not is_flagged:
        fn += 1
    else:
        tn += 1

precision = tp / (tp + fp) if (tp + fp) else 0

```

```

recall = tp / (tp + fn) if (tp + fn) else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0

# Why the false positives happen: CV misclassification and/or extraction
# error pushing a genuinely-consistent claim outside the agent's expected
# range - a real, explainable cause, not a mystery.
fp_causes = []
for claim_id in evaluated_ids:
    is_true_mismatch = bool(truth_df.loc[claim_id, "is_injected_mismatch"])
    is_flagged = claim_id in flagged_ids
    if is_flagged and not is_true_mismatch:
        true_severity = truth_df.loc[claim_id, "true_severity"]
        predicted_severity = vision_preds[claim_id]["predicted_severity"]
        fp_causes.append({
            "claim_id": claim_id,

```

4.11 scripts/load_raw_to_duckdb.py

Why it exists. Where AI outputs become warehouse tables and acquire a contract.

What the module says about itself. Loads every raw output this platform's pipelines produce - the ground truth, both document-extraction approaches, the CV predictions, and the reconciliation flags - into raw DuckDB tables that dbt_project/ then builds the governed staging -> intermediate -> marts layers on top of. This is the same "land raw, then transform in dbt" separation used in every prior dbt-based project in this portfolio.

Opening of the module

scripts/load_raw_to_duckdb.py

```

"""
load_raw_to_duckdb.py
-----

Loads every raw output this platform's pipelines produce - the ground
truth, both document-extraction approaches, the CV predictions, and the
reconciliation flags - into raw DuckDB tables that dbt_project/ then
builds the governed staging -> intermediate -> marts layers on top of.
This is the same "Land raw, then transform in dbt" separation used in
every prior dbt-based project in this portfolio.

Run this AFTER: scripts/generate_claims_dataset.py, document_ai/ocr_extract.py,
document_ai/regex_extraction.py, document_ai/llm_field_extraction.py,
vision/train_damage_classifier.py (+ predict), agents/claim_reconciliation_agent.py

Output: database/warehouse.duckdb (raw.* tables)
"""

import json
from pathlib import Path

import duckdb
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
REPORTS_DIR = ROOT / "reports"
DATABASE_DIR = ROOT / "database"
DATABASE_DIR.mkdir(exist_ok=True)

conn = duckdb.connect(str(DATABASE_DIR / "warehouse.duckdb"))
conn.execute("CREATE SCHEMA IF NOT EXISTS raw")

# ground truth
gt = pd.read_csv(DATA_DIR / "claims_ground_truth.csv")
conn.execute("CREATE OR REPLACE TABLE raw.claims_ground_truth AS SELECT * FROM gt")

# LLM extraction
with open(DATA_DIR / "llm_extracted_fields.json", encoding="utf-8") as f:
    llm_raw = json.load(f)
llm_rows = []
for claim_id, fields in llm_raw.items():
    row = {"claim_id": claim_id}
    row.update(fields)
    llm_rows.append(row)
llm_df = pd.DataFrame(llm_rows)
conn.execute("CREATE OR REPLACE TABLE raw.extraction_llm AS SELECT * FROM llm_df")

# Regex extraction
with open(DATA_DIR / "regex_extracted_fields.json", encoding="utf-8") as f:
    regex_raw = json.load(f)
regex_rows = []
for claim_id, fields in regex_raw.items():
    row = {"claim_id": claim_id}
    row.update(fields)
    regex_rows.append(row)
regex_df = pd.DataFrame(regex_rows)
conn.execute("CREATE OR REPLACE TABLE raw.extraction_regex AS SELECT * FROM regex_df")

```

```

# Vision predictions
with open(REPORTS_DIR / "vision_predictions.json", encoding="utf-8") as f:
    vision_raw = json.load(f)
    vision_rows = [{"claim_id": k, **v} for k, v in vision_raw.items()]
    vision_df = pd.DataFrame(vision_rows)
    conn.execute("CREATE OR REPLACE TABLE raw.vision_predictions AS SELECT * FROM vision_df")

# Reconciliation flags
with open(REPORTS_DIR / "reconciliation_flags.json", encoding="utf-8") as f:
    flags_raw = json.load(f)
    if flags_raw:
        flags_df = pd.DataFrame(flags_raw)
    else:
        flags_df = pd.DataFrame(columns=["claim_id", "predicted_severity", "item_value",

```

4.12 dashboard/streamlit_app.py

Why it exists. The adjuster's console: document, photo, extraction, severity and verdict on one screen.

What the module says about itself. The unified ops dashboard for the AutoClaim Intelligence Platform - four tabs, all reading live from what the rest of the system actually produced (dbt's governed fct_claims table via scripts/compute_metrics.py, never a separate hand-written query):

Public functions in dashboard/streamlit_app.py

Function	What it does
load_json(path, default)	—

Opening of the module

dashboard/streamlit_app.py

```

"""
streamlit_app.py
-----
The unified ops dashboard for the AutoClaim Intelligence Platform - four
tabs, all reading live from what the rest of the system actually
produced (dbt's governed fct_claims table via scripts/compute_metrics.py,
never a separate hand-written query):

    Claims Ops          - the flagged-claims queue an adjuster would work from
    Document AI         - regex baseline vs. LLM extraction accuracy
    Computer Vision     - the damage-severity CNN's confusion matrix
    Data Platform Health - dbt contract/test status, the governed metrics

Run with:
    streamlit run dashboard/streamlit_app.py --server.port 8503
"""

import json
import sys
from pathlib import Path

import duckdb
import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from scripts.compute_metrics import compute_metrics # noqa: E402

REPORTS_DIR = ROOT / "reports"
DB_PATH = ROOT / "database" / "warehouse.duckdb"

st.set_page_config(page_title="AutoClaim Intelligence Platform", layout="wide", page_icon="📊")

def load_json(path, default=None):
    if not path.exists():
        return default
    with open(path, encoding="utf-8") as f:
        return json.load(f)

st.title("📊 AutoClaim Intelligence Platform")
st.caption("Document AI + Computer Vision + a governed dbt semantic layer, unified around "
           "insurance claims processing - reconciled by a cross-modal agent.")

tab_ops, tab_doc, tab_cv, tab_platform = st.tabs(
    ["📄 Claims Ops", "📄 Document AI", "📄 Computer Vision", "📄 Data Platform Health"])

# =====
# TAB 1: Claims Ops
# =====
with tab_ops:
    if not DB_PATH.exists():
        st.info("Run the pipeline first: `python scripts/run_pipeline.py`")
    else:

```

```

metrics = compute_metrics()
k1, k2, k3, k4 = st.columns(4)
k1.metric("Total Claims", int(metrics["claim_count"]))
k2.metric("Total Claimed", f"${metrics['total_claimed_amount']:.0f}")
k3.metric("Flagged Claims", int(metrics["flagged_claim_count"]))
k4.metric("Flagged Rate", f"${metrics['flagged_claim_rate']:.1%}")

st.divider()
st.subheader("🚩 Flagged Claims Queue")
flags = load_json(REPORTS_DIR / "reconciliation_flags.json", default=[])
if flags:
    flags_df = pd.DataFrame(flags)
    st.dataframe(
        flags_df[["claim_id", "predicted_severity", "item_value", "claimed_amount",
                  "claimed_fraction"]],

```

CHAPTER 5

The Damage Classifier

A convolutional network trained from scratch on 258 images, what its 83.9% accuracy hides, and why the direction of its errors matters more than their count.

5.1 The task and the constraint

Four classes - none, minor, moderate, severe - from a photograph of a damaged item. 258 training images, 62 test images, 25 epochs, trained from scratch on a CPU.

WHY FROM SCRATCH RATHER THAN FINE-TUNING A PRETRAINED NETWORK

Fine-tuning a ResNet on ImageNet weights would score better and would demonstrate less. Training from scratch on a small set makes every part of the problem visible: how quickly it overfits, how much the class balance matters, and what 258 images can and cannot support.

It also respects the portfolio's constraint - the whole project must run for free on modest hardware, and a from-scratch CNN on 96-pixel images trains in minutes on a CPU.

The classifier

vision/train_damage_classifier.py

```

"""
train_damage_classifier.py
-----
Trains a small convolutional neural network, from scratch, on the
synthetic damage photos (data/damage_images/{none,minor,moderate,severe}/)
to classify damage severity - the computer vision half of this platform,
kept deliberately small (3 conv blocks, 96x96 grayscale-ish RGB inputs)
so it trains in minutes on CPU with no GPU and no pretrained-model
download, consistent with every other model choice in this portfolio
(no heavy framework where a lighter one does the job on this hardware).

Uses `torchvision`-free image loading (plain PIL + a hand-rolled Dataset)
since torchvision isn't installed on this machine and isn't needed for
something this small.

Split: stratified 80/20 train/test by class, so evaluation reflects a
genuine holdout, not training-set performance.

Output:
  vision/artifacts/damage_classifier.pt - trained model weights
  reports/vision_evaluation.json/.md    - accuracy, confusion matrix, per-class P/R
"""

import json
from collections import defaultdict
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from PIL import Image

ROOT = Path(__file__).resolve().parent.parent
IMG_DIR = ROOT / "data" / "damage_images"
ARTIFACTS_DIR = ROOT / "vision" / "artifacts"
ARTIFACTS_DIR.mkdir(exist_ok=True)
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)

SEED = 5
torch.manual_seed(SEED)
rng = np.random.default_rng(SEED)

CLASSES = ["none", "minor", "moderate", "severe"]
CLASS_TO_IDX = {c: i for i, c in enumerate(CLASSES)}
IMG_SIZE = 96
TEST_FRACTION = 0.2
BATCH_SIZE = 16
N_EPOCHS = 25
LR = 1e-3

def load_dataset():
    paths, labels = [], []
    for cls in CLASSES:
        for p in sorted((IMG_DIR / cls).glob("*.png")):

```



```

        paths.append(p)
        labels.append(CLASS_TO_IDX[cls])
    return paths, np.array(labels)

def stratified_split(paths, labels):
    train_idx, test_idx = [], []
    for cls_idx in range(len(CLASSES)):
        idx = np.where(labels == cls_idx)[0]
        rng.shuffle(idx)
        n_test = max(1, int(len(idx) * TEST_FRACTION))
        test_idx.extend(idx[:n_test])
        train_idx.extend(idx[n_test:])

```

5.2 What a convolution actually does

A convolutional layer slides a small learned filter across the image and records how strongly each position matches it:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) K(i, j)$$

Two properties follow, and they are the reason convolutions rather than fully-connected layers are used on images.

1. **Translation equivariance.** The same filter detects the same feature wherever it appears. A scratch in the corner and a scratch in the centre excite the same filter.
2. **Parameter sharing.** One 3x3 filter is nine weights regardless of image size. A fully-connected layer on a 96x96 image would need 9,216 weights per unit for the first layer alone - and with 258 training images it would memorise them.

DEPTH BUILDS A HIERARCHY NOBODY DESIGNED

Early filters learn edges and colour gradients because those are the features that best explain local pixel structure. Later layers compose them into corners, textures and eventually into whatever distinguishes a dent from a crack.

Nothing specifies this. It emerges from gradient descent, in the same way the latent factors of Project 7's matrix factorisation emerge from the decomposition - the architecture constrains what *can* be learned and the data decides what is.

5.3 83.9% accuracy, and what it conceals

6.1 The confusion matrix, from the real evaluation

True \ Predicted	none	minor	moderate	severe
none (8)	8	0	0	0
minor (22)	0	21	1	0
moderate (21)	0	6	14	1
severe (11)	0	0	2	9

6.2 Per-class performance

Class	Precision	Recall	Support
none	1.000	1.000	8
minor	0.778	0.955	22
moderate	0.824	0.667	21
severe	0.900	0.818	11

ONE CLASS CARRIES MOST OF THE ERROR

Overall accuracy is 83.9%. Moderate damage is recalled at **66.7%** - a third of genuinely moderate claims are called something else, and six of the seven errors call them *minor*.

none is perfect at 8 for 8, which is unsurprising: an undamaged item looks obviously different from a damaged one. The hard boundary is between adjacent severities, which is exactly where a human adjuster would also hesitate - and where the money actually differs.

5.4 The errors have a direction

Count the off-diagonal cells by whether the prediction is milder or more severe than the truth.

6.3 The direction of the classifier's mistakes

Direction	Cells	Count
Under-calls severity (predicted milder)	moderate→minor 6, severe→moderate 2	8
Over-calls severity (predicted worse)	minor→moderate 1, moderate→severe 1	2
	total errors	10 of 62

FOUR OUT OF FIVE ERRORS RUN IN THE SAME DIRECTION, AND IT IS THE HARMFUL ONE

In a claims system, calling damage milder than it is means the claim is questioned, reduced, or rejected. Calling it worse means an overpayment. These are not symmetric costs and they do not fall on the same party: the claimant absorbs the first and the insurer absorbs the second.

A model with an 8:2 bias toward under-severity systematically disadvantages claimants.

No summary accuracy figure reveals that - 83.9% is compatible with errors in any direction, and the direction is the part with a person on the other end of it.

Why it happens, and what would fix it

Minor is the largest class in training (22 of 62 in test, and proportionally similar in train). A model uncertain between minor and moderate does best on average by guessing the more common one, so the class imbalance produces the bias directly.

Two fixes, neither implemented here. **Class weighting** in the loss makes a moderate misclassified as minor cost more than the reverse. **An ordinal loss** would recognise that these classes are ordered - confusing minor with severe should cost more than confusing minor with moderate, and a plain cross-entropy treats all four classes as unrelated labels.

THE ORDINAL POINT IS WORTH HOLDING ONTO

`none < minor < moderate < severe` is a scale, not a set of names. Cross-entropy throws that structure away: it learns four independent categories and is equally penalised for every kind of mistake.

Treating an ordered target as unordered is one of the most common modelling errors in applied work, and it costs exactly the information that would have told the model its moderate-called-minor errors are worse than its moderate-called-severe ones.

CHAPTER 6

Cross-Modal Reconciliation, and Where Its Errors Come From

The consistency check, its perfect recall, its 65.5% precision, and the tracing of eight false positives to a cause two stages upstream.

6.1 The check

For each claim, compute what fraction of the item's value is being claimed, and ask whether that fraction is plausible for the damage severity the photograph shows.

$$f = \frac{\text{claimed amount}}{\text{item value}}, \quad \text{flag if } f \notin [l_s, h_s]$$

8.1 The agent's bands are deliberately wider than the generator's

Severity	Agent's expected band	Generator's actual band
none	(0.00, 0.10)	(0.00, 0.05)
minor	(0.05, 0.30)	(0.08, 0.25)
moderate	(0.25, 0.65)	(0.30, 0.60)
severe	(0.60, 1.05)	(0.65, 1.00)

THE BANDS WERE SET INDEPENDENTLY AND MADE MORE PERMISSIVE ON PURPOSE

The agent's bands are its own business-reasonableness judgement, not a copy of the generator's. They are wider at both ends, so a claim sitting anywhere inside the generator's legitimate range is comfortably inside the agent's.

That gap is deliberate: **the agent has to clear a real bar rather than replay the answer key.** Had the bands been copied, perfect performance would be guaranteed and meaningless - the same circularity as the regex in Chapter 4.

The reconciliation check

agents/claim_reconciliation_agent.py

```
def check_claim(severity: str, item_value: float, claimed_amount: float) -> dict:
    if item_value <= 0:
        return {"flagged": False, "reason": "item_value not available for reconciliation"}
    fraction = claimed_amount / item_value
    lo, hi = EXPECTED_FRACTION_RANGE.get(severity, (0.0, 1.0))
    flagged = not (lo <= fraction <= hi)
    return {"flagged": flagged, "fraction": round(fraction, 3), "expected_range": (lo, hi)}

def explain_flag(claim_id, severity, item_value, claimed_amount, fraction, lo, hi) -> str:
```

6.2 Perfect recall, and what it is worth

8.2 The reconciliation confusion matrix, n = 80

	Actually inconsistent	Actually legitimate
Flagged	19	10
Not flagged	0	51

$$\text{recall} = \frac{19}{19} = 1.000, \quad \text{precision} = \frac{19}{29} = 0.655$$

RECALL OF 1.000 WAS CLOSE TO GUARANTEED BY THE GENERATOR'S DESIGN

The generator creates an inconsistent claim by drawing the amount from a *different severity's* band entirely - a severe-band amount on an undamaged item, or a none-band amount on a severe one. Those land enormously far outside the expected range.

Catching them is not a demonstration of a sensitive detector; it is a demonstration that the injected anomalies are large. **The honest claim is that the check catches gross mismatches, which is what it was built for** - and it says nothing about a subtle over-claim of 15%, which is what real claim fraud looks like.

6.3 Tracing the false positives

Ten legitimate claims were flagged. The evaluation report records, for each one, whether the vision model got the severity right.

8.3 Every false positive, with the vision model's verdict

Claim	True severity	Predicted	CV correct?
CLAIM_10002	moderate	moderate	yes
CLAIM_10003	moderate	minor	no
CLAIM_10005	none	none	yes
CLAIM_10015	minor	minor	yes
CLAIM_10016	moderate	moderate	yes
CLAIM_10030	minor	minor	yes
CLAIM_10055	moderate	moderate	yes
CLAIM_10057	moderate	moderate	yes
CLAIM_10064	minor	minor	yes
CLAIM_10069	moderate	minor	no

EIGHT OF TEN HAD A CORRECT SEVERITY

Only CLAIM_10003 and CLAIM_10069 involve a vision error. In the other eight the photograph was classified correctly, the claim was legitimate, and the flagging rule is sound - and the claim was flagged anyway.

Improving the vision model - the obvious response to 65.5% precision - would fix 20% of the false positives. Something else is causing the rest.

6.4 The actual cause

Take CLAIM_10002. Its true fraction, computed from the ground-truth claim record, is **0.465**. Its severity is moderate. The expected band for moderate is (0.25, 0.65). **0.465 sits comfortably inside it**, so the rule should not have fired.

Now look at what the agent was actually given:

The record as the agent received it

```
CLAIM_10002:
  item_value      = 90.73
  claimed_amount  = 90.73    <-- identical
  fraction        = 1.000
  expected range  = (0.25, 0.65)
  -> FLAGGED
```

THE EXTRACTOR RETURNED THE SAME NUMBER FOR TWO DIFFERENT FIELDS

Item value and claimed amount both came back as \$90.73, producing a fraction of exactly 1.000 against a true fraction of 0.465. The agent's input was corrupt, so its output was wrong - correctly, given what it was told.

The reconciliation agent consumes **LLM-extracted** fields, and the LLM's measured accuracy on those two fields is **0.900** for item value and **0.9625** for claimed amount. Roughly one claim in ten arrives at the reconciliation stage with a wrong item value.

The arithmetic of the whole chain

The reconciliation decision is correct only if all three of its inputs are correct:

$$0.900 \times 0.9625 \times 0.839 = 0.727$$

So about **27% of claims** reach the final decision with at least one corrupted input. Not all of those produce a wrong flag - a wrong value can still land inside the band - which is why the observed spurious flag rate is 10 of 80 rather than 22 of 80.

THIS IS THE FINDING, AND IT GENERALISES

Three components at 90%, 96% and 84% are all individually respectable. Nobody reviewing them one at a time would flag a problem. Composed, they leave a quarter of the pipeline's decisions resting on at least one bad input.

Multi-stage AI pipelines degrade multiplicatively, and no single stage's metrics reveal it.

For n stages each at accuracy a , the chain is a^n : five stages at 95% is 77%; ten is 60%.

What would actually improve precision here

1. **Cross-check the extraction.** The regex and the LLM extract the same fields. Where they disagree, route to a human. On this data the regex is perfect, so this alone would eliminate the eight false positives - and on real documents it becomes a genuine ensemble rather than a free win.
2. **Reject impossible field combinations.** `item_value == claimed_amount` to the cent is a suspicious coincidence, not a business fact. A cheap validity rule catches exactly this failure.
3. **Propagate confidence.** If the extractor reported low confidence on item value, the reconciliation agent should widen its band or abstain rather than flag. Currently every input is treated as certain.

NONE OF THOSE THREE IS 'IMPROVE THE VISION MODEL'

Which is the practical point. The stage with the worst-looking headline metric - 83.9% - is not where the errors come from. Attribution across a pipeline requires tracing individual failures to their source, and the per-stage metrics actively mislead about where to spend effort.

CHAPTER 7

The Semantic Layer and Data Contracts

Where AI outputs become governed metrics, why a contract at that boundary matters, and what lineage is for.

7.1 AI outputs are data, and data needs governance

The document extractor, the vision model and the reconciliation agent all produce structured output that somebody will build a report on. At that moment their predictions stop being model outputs and become columns in a warehouse.

The dbt layer in this project is where that transition is made explicit: extracted fields, predicted severities and reconciliation flags are loaded, tested and modelled into metrics the dashboard reads.

A CONTRACT AT THE AI BOUNDARY IS DIFFERENT FROM AN ORDINARY SCHEMA TEST

A source system that changes its output is a bug you file against another team. A *model* that changes its output is doing exactly what it is supposed to do - the world moved, or it was retrained, and the distribution of its predictions shifted.

So the contract has to encode more than types. It has to encode the assumptions downstream metrics rest on: that `predicted_severity` takes one of exactly four values, that `claimed_amount` is positive, that the flag rate stays within a plausible band. The last of those is a *distributional* contract, and it is the one Project 7's drift monitor was doing in a different guise.

7.2 Lineage, and the question it answers

Because dbt derives the graph from `ref()`, the path from a dashboard number back to the model that produced it is recoverable automatically.

The question that makes this worth having is the one asked after an incident: *the vision model was retrained on Tuesday - which reported figures moved because of it?* With lineage that is a graph traversal. Without it, it is an afternoon of grepping and a guess.

BLAST RADIUS IS THE USEFUL FRAMING

Every model in a platform has a set of downstream consumers, and the size of that set is the cost of changing it. A vision model feeding one dashboard panel can be swapped freely; one feeding a metric that appears in a regulatory return cannot.

Project 13 and **Project 15** build this out properly - governed metric definitions, and a metric-regression test that proved a single 'clarifying' definition change silently restated 33 of 50 published figures.

CHAPTER 8

Worked Examples

The reconciliation arithmetic on real claims, the compounding of stage accuracies, and what each number means.

8.1 Example 1 - A correctly flagged claim

CLAIM_10000, from the real flag file. The photo was classified **moderate**, so the agent's expected band is (0.25, 0.65).

10.1 A genuine mismatch

Quantity	Value
Extracted item value	\$922.27
Extracted claimed amount	\$6.82
Fraction claimed	$6.82 / 922.27 = 0.007$
Expected band for moderate	(0.25, 0.65)
Verdict	0.007 < 0.25 - flagged

A moderately damaged item worth \$922 with a claim of \$6.82. Either the photo is of the wrong item, or the amount was mistyped, or something stranger. It is exactly the kind of case an adjuster should see, and the flag is correct.

NOTE WHICH DIRECTION THIS MISMATCH RUNS

This is a claim for far *less* than the damage suggests, which is not fraud - nobody defrauds an insurer by under-claiming. It is almost certainly a data-entry error, and flagging it protects the claimant rather than the insurer.

A reconciliation check that only fired on over-claims would be a fraud detector. Firing in both directions makes it a data-quality control, which is the more useful thing to have built.

8.2 Example 2 - A false positive, and its real cause

CLAIM_10002. The vision model was **correct** - true severity moderate, predicted moderate. The claim was **not** deliberately made inconsistent by the generator. And it was flagged.

10.2 Where the false positive came from

Quantity	What the agent saw	Ground truth
Severity	moderate (correct)	moderate
Item value	\$90.73	not \$90.73
Claimed amount	\$90.73	-
Fraction	1.000	0.465
Band (0.25, 0.65)	1.000 is outside - flagged	0.465 is inside - should not flag

THE EXTRACTOR RETURNED THE SAME NUMBER FOR TWO DIFFERENT FIELDS

Item value and claimed amount both came back as \$90.73, giving a fraction of exactly 1.000. The true fraction is 0.465, which sits comfortably inside the expected band.

The vision model was right. The flagging rule was right. The claim was legitimate. It was flagged anyway, because an upstream extraction error corrupted the input. Chapter 8 shows this is the cause of most of the false positives, not the two the evaluation report attributes to vision errors.

8.3 Example 3 - Compounding stage accuracy

The reconciliation check is correct only if all three of its inputs are correct. Assuming independence, the probability of that is the product:

$$\begin{aligned}
 P(\text{all correct}) &= P(\text{item value}) \times P(\text{claimed amount}) \times P(\text{severity}) \\
 &= 0.900 \times 0.9625 \times 0.839 = 0.727
 \end{aligned}$$

10.3 Three good stages, one mediocre pipeline

Stage	Measured accuracy	Running product
Item value (LLM extraction)	0.900	0.900
Claimed amount (LLM extraction)	0.9625	0.866
Severity (vision)	0.839	0.727

EVERY STAGE LOOKS ACCEPTABLE AND THE CHAIN DOES NOT

Nobody would call 90%, 96% or 84% a failing component. Composed, they leave roughly **27% of claims with at least one corrupted input** to the reconciliation decision.

Not every corrupted input produces a wrong flag - some wrong values still land inside the band - which is why the observed spurious flag rate is 10 of 80 rather than 22 of 80. But the ceiling is set by the product, and no amount of improving the final stage can raise it.

The general rule

For a pipeline of n stages each with accuracy a , end-to-end accuracy is a^n . At $a = 0.95$ and $n = 5$ that is 77%; at $n = 10$ it is 60%. **Multi-stage AI pipelines degrade multiplicatively, and the degradation is invisible in any single stage's metrics.**

8.4 Example 4 - Reading a perfect score

Two numbers in this project are exactly 1.000, and both deserve the same suspicion.

10.4 Two perfect scores and their explanations

Metric	Value	Why it is 1.000
Regex extraction, all six fields	1.0000	the regex was written against the template that generated the PDFs
Reconciliation recall	1.0000	the generator's inconsistent claims are far outside the bands

A PERFECT SCORE IS A CLAIM ABOUT THE TEST, NOT ABOUT THE SYSTEM

Neither number is wrong. Both are uninformative about how the system would behave on anything other than exactly this data.

The portfolio's standing rule - **a suspiciously good result is a bug report** - applies. In Project 9 a PII detector scoring 1.000 turned out to be scoring its own training distribution and fell to 0.20 on a held-out challenge set. The same investigation is what Chapter 4 does to the regex result here.

CHAPTER 9

Running the Project, Step by Step

Every terminal command needed to go from a fresh clone to a finished run, what each one does, and what to do when one of them fails.

9.1 Prerequisites

You need Python 3.11 or newer and Git. Verify both before starting - most reported 'the project does not work' turns out to be an environment that was never activated.

Check your toolchain

```
python --version
git --version
```

THIS PROJECT HAS ONE OPTIONAL LOCAL-LLM STAGE

Install Ollama and pull the model if you want the generative feature to run. It is optional: the project completes without it, and the deterministic control arm still produces results.

Optional - only for the LLM stage

```
ollama pull qwen2.5:0.5b
ollama list
```

9.2 Step 1 - Clone and enter the repository

Get the code

```
git clone https://github.com/Nikhil201716/08-AutoClaim-Intelligence-Platform.git
cd 08-AutoClaim-Intelligence-Platform
```

9.3 Step 2 - Create an isolated environment

A virtual environment keeps this project's dependencies from colliding with anything else on your machine.

Create and activate

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

IF POWERSHELL REFUSES TO RUN THE ACTIVATION SCRIPT

Windows blocks unsigned scripts by default. Allow them for your own user account only:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

9.4 Step 3 - Install dependencies

Install

```
pip install -r requirements.txt
```

9.5 Step 4 - Run the pipeline, stage by stage

Each command below produces the input the next one consumes. Run them in this order:

The full pipeline in order

```
python scripts/generate_claims_dataset.py
python document_ai/ocr_extract.py
python document_ai/regex_extraction.py
python document_ai/llm_field_extraction.py
python document_ai/evaluate_extraction.py
python vision/train_damage_classifier.py
python vision/predict.py
python agents/claim_reconciliation_agent.py
python scripts/evaluate_reconciliation.py
```

What each stage does:

1. `scripts/generate_claims_dataset.py` - writes the seeded claim records, renders the claim PDFs and the damage images, and marks which claims are deliberately inconsistent
2. `document_ai/ocr_extract.py` - runs OCR over every claim PDF and writes `data/ocr_raw_text.json`
3. `document_ai/regex_extraction.py` - extracts the six fields by pattern matching against the known template

4. `document_ai/llm_field_extraction.py` - extracts the same six fields with the local model, without being shown the layout
5. `document_ai/evaluate_extraction.py` - scores both extractors against ground truth, field by field
6. `vision/train_damage_classifier.py` - trains the CNN from scratch on the damage images and writes the confusion matrix
7. `vision/predict.py` - classifies every claim image, producing the severities the agent consumes
8. `agents/claim_reconciliation_agent.py` - runs the cross-modal consistency check and writes the flagged claims
9. `scripts/evaluate_reconciliation.py` - grades the flags against the injected inconsistencies and attributes each false positive

EXPECTED OUTPUT

Expect regex extraction at 1.0000 and LLM at 0.9750 - and read Chapter 4 before concluding anything from that gap. Vision accuracy should land near 83.9%; because the CNN is trained from scratch the exact figure moves a little between runs, but the confusion matrix should keep its shape, with moderate the weakest class and most errors calling damage milder than it is. Reconciliation should give recall 1.000 and precision near 0.655, with 8 of the 10 false positives having a correct severity. The LLM stages need Ollama; the regex, vision and evaluation stages do not.

9.6 Run everything in one command

Executes every stage above in order. Use this for a clean rebuild.

```
python scripts/run_pipeline.py
```

9.7 Reproduce the false-positive attribution

Should print 8 of 10 - the finding of Chapter 8. If it prints something else, the vision model has been retrained and the attribution has moved.

```
python -c "import json; d=json.load(open('reports/reconciliation_evaluation.json'));
fp=d['false_positive_detail']; print('correct CV:', sum(1 for f in fp if f['cv_correct']), 'of',
len(fp))"
```

9.8 Inspect the extraction that caused CLAIM_10002

Shows item_value and claimed_amount both extracted as the same number - the corrupt input behind that false positive.

```
python -c "import json; print(json.load(open('data/llm_extracted_fields.json'))['CLAIM_10002'])"
```

9.9 Compare the two extractors directly

The disagreement rate is the input to the cross-check policy proposed in Chapter 8 and exercise 8.

```
python -c "import json; r=json.load(open('data/regex_extracted_fields.json'));
l=json.load(open('data/llm_extracted_fields.json')); print(sum(1 for k in l if r.get(k,
{}).get('item_value')!=l[k].get('item_value')), 'disagreements on item_value')"
```

9.10 Launch the adjuster's console

Opens on <http://localhost:8501> with the document, the photo, the extracted fields, the predicted severity and the reconciliation verdict per claim.

```
streamlit run dashboard/streamlit_app.py
```

9.11 Command reference card

Every command from this chapter in one block, for when you already know what you want.

Complete command reference

```
# --- setup (once) ---
git clone https://github.com/Nikhil201716/08-AutoClaim-Intelligence-Platform.git
cd 08-AutoClaim-Intelligence-Platform
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows
source .venv/bin/activate          # macOS / Linux
pip install -r requirements.txt

# --- run ---
python scripts/generate_claims_dataset.py
python document_ai/ocr_extract.py
python document_ai/regex_extraction.py
python document_ai/llm_field_extraction.py
python document_ai/evaluate_extraction.py
python vision/train_damage_classifier.py
python vision/predict.py
python agents/claim_reconciliation_agent.py
python scripts/evaluate_reconciliation.py

# --- extras ---
python scripts/run_pipeline.py
python -c "import json; d=json.load(open('reports/reconciliation_evaluation.json'));
fp=d['false_positive_detail']; print('correct CV:', sum(1 for f in fp if f['cv_correct']), 'of',
len(fp))"
python -c "import json; print(json.load(open('data/llm_extracted_fields.json'))['CLAIM_10002'])"
python -c "import json; r=json.load(open('data/regex_extracted_fields.json'));
l=json.load(open('data/llm_extracted_fields.json')); print(sum(1 for k in l if r.get(k,
{}).get('item_value')!=l[k].get('item_value')), 'disagreements on item_value')"
streamlit run dashboard/streamlit_app.py

# --- finish ---
deactivate
```

9.12 Troubleshooting

Common problems and their fixes

Symptom	Cause	Fix
<code>ModuleNotFoundError</code>	environment not activated	re-run the activate command from Step 2
<code>FileNotFoundError</code> on a data file	an earlier stage was skipped	run the stages in the documented order
Numbers differ from this book	a seed or parameter was changed	check the constants at the top of the generator
command not found	dependency missing from this env	<code>pip install -r requirements.txt</code> with the env active
Permission denied writing output	directory created by another user or process	delete the generated output directory and re-run

CHAPTER 10

The Adjuster's Console

What a claims reviewer needs on screen, and why the flagged queue is the whole product.

10.1 One claim, four sources of evidence

The dashboard puts the claim document, the damage photograph, the extracted fields, the predicted severity and the reconciliation verdict on one screen - which is the actual job. An adjuster's work is comparing sources, and the platform's value is doing the comparison first and surfacing the disagreements.

Launch the console

```
streamlit run dashboard/streamlit_app.py
```

10.2 Why the explanation is generated and the decision is not

The flag is a threshold check in code. The one-sentence explanation an adjuster reads is generated by the local model from the numbers the check already produced.

THE FOURTH APPEARANCE OF THE SAME ARCHITECTURE

Project 5 grounds answers in retrieved documents. Project 6 computes reason strings before any model sees them. Project 7 constrains the action agent to a closed set. Here the reconciliation verdict is arithmetic and the model only narrates it.

The pattern is worth naming because it is the single most transferable idea in this portfolio: **put the correctness in code and the language in the model.** It is what makes a 0.5-billion-parameter model - which cannot be trusted to reason about money - safe to place in front of a claims adjuster.

It also means the explanation cannot contradict the decision, because it is generated *from* the decision. A system where the model decides and then explains can produce a flag for one reason and a justification citing another, with nothing detecting the divergence - the same argument Project 6 makes about a customer's right to know why their card was declined.

CHAPTER 11

Results, and What They Do Not Establish

Every measured figure, and the limits of each one.

11.1 The numbers

11.1 Every measured figure

Metric	Value
Claims with PDFs and images	320 regex-evaluated, 80 LLM-evaluated
Regex extraction accuracy	1.0000 across all six fields
LLM extraction accuracy	0.9750 overall
Vision training / test images	258 / 62
Vision test accuracy	83.87%
Vision error direction	8 under-severity, 2 over-severity
Reconciliation recall	1.000 (19 of 19)
Reconciliation precision	0.655 (19 of 29)
False positives with a correct severity	8 of 10
Implied all-inputs-correct rate	0.727

11.2 The four findings worth defending

1. The regex's perfect score is circular

It was written against the template that generated the documents, so 1.0000 was determined before any test ran. The conclusion it supports is narrow: on documents matching a known fixed template, prefer the regex. It says nothing about documents that vary.

2. The vision model's errors are directionally biased

Eight of ten errors call damage milder than it is. In a claims system that means systematic under-payment, borne by claimants. Overall accuracy of 83.9% is compatible with errors in any direction and conceals which one.

3. Eight of ten false positives are not the vision model's fault

They arrive with a correct severity, a legitimate claim and a sound rule. Improving the CNN addresses at most 20% of the precision problem.

4. The cause is upstream extraction error, and it compounds

CLAIM_10002 was flagged because the extractor returned the same value for item value and claimed amount. Across the chain, $0.900 \times 0.9625 \times 0.839 = 0.727$ - about a quarter of decisions rest on at least one bad input.

11.3 What this does not establish

1. **That the regex is better than the LLM.** The comparison was conducted entirely on documents matching the regex's template. The informative experiment - vary the layout and find the crossover - was not run.
2. **That the vision model generalises.** 258 training images of synthetically rendered damage marks, not photographs. The 83.9% describes this rendering, not real claim photos.
3. **That reconciliation recall would hold on subtle cases.** The injected inconsistencies are gross - an amount drawn from a completely different severity's band. A 15% over-claim would not be caught, and was never tested.
4. **That 0.727 is the true all-correct rate.** It assumes the three stages fail independently. They may not - a hard-to-read document is plausibly also a hard-to-photograph item.
5. **That the precision figure is precisely estimated.** 19 of 29 on 80 claims; the confidence interval is wide.
6. **That the semantic layer's contracts would catch a model regression.** They are structural tests; a model whose output distribution shifts while remaining well-typed passes them all.

THE MOST USEFUL SENTENCE IN THIS CHAPTER

Every headline number in this project is defensible and every one of them is less impressive than it first appears. The regex's 1.000 is circular, the reconciliation's 1.000 is a property of the injected anomalies' size, and the 83.9% conceals a directional bias that matters more than the percentage.

Reading a result well is mostly asking what would have had to be true for it to come out differently.

CHAPTER 12

Exercises, with Solutions

Problems requiring the chapters, with worked answers.

12.1 Exercises

1. The regex scores 1.0000 and the LLM 0.9750. Explain why this does not show the regex is better, and design the experiment that would settle it.
2. A pipeline has five stages at 95% accuracy each. What is the end-to-end rate? How many stages before it drops below 50%?
3. CLAIM_10002 has a true fraction of 0.465 and a moderate severity, and the band is (0.25, 0.65). Explain why it was flagged.
4. The vision model makes 8 under-severity and 2 over-severity errors. Why does the direction matter more than the count, and who bears each cost?
5. Reconciliation recall is 1.000. State precisely what that does and does not demonstrate.
6. Propose three changes that would improve reconciliation precision, none of which is 'improve the vision model'. Rank them by expected effect.
7. Cross-entropy treats none/minor/moderate/severe as four unrelated labels. What information is lost, and what would an ordinal loss do differently?
8. An LLM extractor and a regex extractor disagree on a field. Design the policy for what happens next, and say what it costs.

12.2 Solutions

1. The rigged comparison

The PDFs are generated from a fixed template and the regex patterns were written against that template, so the regex is being scored on documents it was fitted to - the result was determined before any test ran. The settling experiment: generate documents with reordered fields, varied labels ('Item Value' / 'Declared Value'), and injected OCR noise, then measure both methods as variation increases. The finding would be the *crossover point* - how much variation it takes for the LLM to win - which is the number a real deployment needs.

2. Compounding

$0.95^5 = 0.774$, so 77.4%. For below 50% you need $0.95^n < 0.5$, giving $n > \ln(0.5)/\ln(0.95) = 13.5$ - so **14 stages**. Every one of those stages looks excellent in isolation, and the pipeline is worse than a coin flip.

3. Why CLAIM_10002 was flagged

The agent never saw 0.465. The LLM extractor returned \$90.73 for both item value and claimed amount, giving a fraction of 1.000, which is outside (0.25, 0.65). The vision model was right, the rule was right, the claim was legitimate - the *input* was corrupt. This is error propagation, and it is invisible from the reconciliation stage's own metrics.

4. The direction of vision errors

Under-calling severity means a claim is questioned, reduced or rejected - the cost falls on the **claimant**. Over-calling means an overpayment - the cost falls on the **insurer**. With 8 errors in one direction and 2 in the other, the system systematically disadvantages claimants, and a person is on the other end of each one. The count is a performance figure; the direction is a fairness question, and only the second has anyone to answer to.

5. What perfect recall shows

It shows that **every deliberately-injected inconsistency was gross enough to fall outside the expected band** - which follows from the generator drawing those amounts from an entirely different severity's range. It demonstrates the check catches large mismatches. It does not demonstrate sensitivity: a 15% over-claim on a moderate item stays well inside (0.25, 0.65) and would be missed, and no such case was tested.

6. Three ways to improve precision

(a) Cross-check regex against LLM extraction and route disagreements to a human - highest expected effect, since on this data it would eliminate all eight extraction-caused false positives. **(b) Reject impossible field combinations** - `item_value == claimed_amount` to the cent is a coincidence, not a fact; cheap and catches exactly the observed failure. **(c) Propagate extraction confidence** so the agent widens its band or abstains on low-confidence inputs - the most principled and the most work.

7. What cross-entropy throws away

The *order*. `none < minor < moderate < severe` is a scale, and cross-entropy treats it as four unrelated categories: confusing minor with severe is penalised identically to confusing minor with moderate. An ordinal loss penalises by distance on the scale, so the model learns that adjacent confusions are cheap and distant ones are expensive - which is both true and directly aligned with the money at stake.

8. The disagreement policy

Treat a disagreement as a low-confidence extraction: do not pick a winner. Either route the claim to a human, or pass both values downstream so the reconciliation agent can abstain rather than flag. The cost is throughput - every disagreement becomes manual work - so the policy is only viable if disagreements are rare, which makes measuring the disagreement rate the first thing to do. Silently preferring one extractor gets you back to a single point of failure with extra machinery.

CHAPTER 13

Appendix

Repository map, glossary, and reproducibility notes.

13.1 A - Repository map

Where everything lives

Path	Purpose
scripts/generate_claims_dataset.py	seeded claims, PDFs, images and injected inconsistencies
document_ai/ocr_extract.py	PDF to raw text
document_ai/regex_extraction.py	template-matched field extraction
document_ai/llm_field_extraction.py	model-based field extraction
document_ai/evaluate_extraction.py	field-by-field scoring of both extractors
vision/train_damage_classifier.py	the from-scratch CNN
vision/predict.py	severity inference over claim images
agents/claim_reconciliation_agent.py	the cross-modal check and its bands
scripts/evaluate_reconciliation.py	grading and false-positive attribution
scripts/load_raw_to_duckdb.py	AI outputs into the warehouse
scripts/compute_metrics.py	the semantic-layer metrics
dashboard/streamlit_app.py	the adjuster's console
reports/extraction_evaluation.json	regex versus LLM, by field
reports/vision_evaluation.json	confusion matrix and per-class metrics
reports/reconciliation_evaluation.json	the confusion matrix and per-claim attribution

13.2 B - Glossary

Terms used in this book

Term	Meaning
Seeded generator	a synthetic data producer whose random draws are reproducible from a fixed starting value
Ground truth	the mechanism deliberately injected into synthetic data, used only to score an analysis - never to make a decision
Control arm / baseline	a simple alternative a new method must beat before it is worth its complexity
Point-in-time correctness	computing a feature using only information that existed at the moment being modelled
Leakage	information from the future or from the target entering a feature, inflating every offline metric
Reproducibility checksum	a hash of generated output, compared across a fresh process to prove determinism
Multimodal	a system whose inputs are of different kinds - here text and images
OCR	optical character recognition - turning an image of text into text
Template matching	extracting fields by their known position or label pattern
Circular evaluation	testing a method against data produced by the same assumptions it encodes
Convolution	sliding a learned filter across an image to detect a feature anywhere in it
Translation equivariance	detecting a feature regardless of where it appears
Parameter sharing	reusing one filter's weights across every position
Ordinal target	a label with a meaningful order, like none < minor < moderate < severe
Cross-entropy	the standard classification loss; treats classes as unordered
Class imbalance	unequal class frequencies, which bias a model toward the common class
Directional bias	errors that run predominantly one way on an ordered scale
Cross-modal reconciliation	checking that two different modalities tell the same story
Error propagation	an upstream mistake corrupting every downstream stage
Compounding accuracy	a chain's accuracy being the product of its stages'
Attribution	tracing an observed failure to the stage that actually caused it
Data contract	an enforced agreement about the shape and content of data at a boundary

Term	Meaning
Lineage	the derivable path from a reported figure back to what produced it

13.3 C - Reproducibility

Every dataset in this project is produced by a seeded generator, so the whole pipeline can be rebuilt from nothing and should produce byte-identical output.

A FIXED SEED IS A CLAIM, NOT A GUARANTEE

Two projects in this portfolio shipped generators with fixed seeds that were still not reproducible across runs. The causes were `rng.choice(list(a_set))` and `abs(hash(some_string))` - both of which are perfectly stable *within* one process and vary between processes, because Python randomises string hashing per interpreter run.

Neither is visible from reading the code, from the seed, or from the output looking sensible. Both were caught only by hashing the generated files and comparing across a *fresh* process. The defences used throughout this portfolio are to sort any collection before it feeds a random draw, and to verify by checksum in a new process rather than trusting the seed.

13.4 D - Where this leads next

This project introduces multimodal AI and the error-propagation problem. The thread continues: **Project 9** confronts the circular-evaluation trap head on - a PII detector that scored 1.000 and fell to 0.20 on a held-out challenge set, which is precisely the investigation Chapter 4 argues the regex result needs. **Project 11** builds the fairness machinery for the directional-bias problem of Chapter 6, where an error's direction matters more than its rate. **Project 12** compares retrieval strategies rigorously enough to find a query class where one scores 0.000. **Project 13** and **Project 15** build the governed semantic layer this project only gestures at, including a metric-regression test that caught a definition change silently restating 33 of 50 published figures.

The three most valuable improvements to *this* project, in order: cross-check the regex and LLM extractions and route disagreements to a human, which would eliminate all eight extraction-caused false positives; vary the document template and measure where the regex stops winning, which converts Chapter 4's rigged comparison into a real finding; and switch the vision model to a class-weighted ordinal loss so its errors stop running systematically against claimants.